



EXPLORING JAVASCRIPT CLOSURES

SWIPE TO LEARN MORE

01

WHAT IS JAVASCRIPT CLOSURES

JavaScript closures are functions that have access to the outer function's scope even after the outer function has finished executing. They are created whenever a function is defined within another function.

```
// Example:  
function outerFunction() {  
  const outerVariable = 'I am an outer variable';  
  function innerFunction() {  
    console.log(outerVariable);  
  }  
  return innerFunction;  
}  
const closureExample = outerFunction();
```


02

WHY THEY ARE IMPORTANT?

Closures allow us to create private variables and methods within functions, preventing external access and ensuring data integrity.

```
// Example:  
function createCounter() {  
  let count = 0;  
  return function() {  
    return ++count;  
  };  
}  
const counter = createCounter();
```

03

HOW DO CLOSURES WORK?

Closures retain references to the variables from their lexical scope, allowing functions to access and manipulate these variables even after the outer function has returned.

```
// Example:  
function createMultiplier(x) {  
  return function(y) {  
    return x * y;  
  };  
}  
const double = createMultiplier(2);
```


04

WHAT PROBLEMS CAN CLOSURES SOLVE?

Closures provide elegant solutions to complex problems, making code more readable, maintainable, and efficient.
javascript.

```
// Example:
function fetchData(url) {
  return function(callback) {
    fetch(url)
      .then(response => response.json())
      .then(data => callback(data))
      .catch(error => console.error(error));
  };
}
const getData = fetchData('https://api.example.com/data');
```

05

HOW CAN CLOSURES ENHANCE FUNCTIONALITY?

Closures allow functions to "remember" their lexical scope, enabling them to access variables from their outer scope even after the outer function has finished executing.

```
// Example:  
function createLogger(prefix) {  
  return function(message) {  
    console.log(`[${prefix}] ${message}`);  
  };  
}  
const logInfo = createLogger('INFO');
```


06

WHEN WE SHOULD USE CLOSURES?

Understanding when and how to use closures effectively can lead to cleaner, more maintainable code and improved application performance.

```
// Example:
function memoize(func) {
  const cache = {};
  return function(arg) {
    if (cache[arg]) {
      return cache[arg];
    } else {
      const result = func(arg);
      cache[arg] = result;
      return result;
    }
  };
}

const memoizedFunction = memoize(someExpensiveOperation);
```

07

HOW DO CLOSURES SUPPORT ASYNCHRONOUS JAVASCRIPT?

Closures play a crucial role in asynchronous JavaScript, enabling functions to access variables from their lexical scope even when executed asynchronously.

```
// Example:
function fetchData(url) {
  return function(callback) {
    fetch(url)
      .then(response => response.json())
      .then(data => callback(data))
      .catch(error => console.error(error));
  };
}
const getData = fetchData('https://api.example.com/data');
```


08

JAVASCRIPT CLOSURES ARE A POWERFUL FEATURE

that enhances the functionality and flexibility of JavaScript programming. Mastery of closures can lead to more efficient, maintainable, and scalable code.

```
// Example:  
// Recapitulating a closure from Slide 2  
const counter = createCounter();
```



**DOUBLE TAP
IF YOU'RE
INSPIRED!**

FOLLOW ME FOR MORE INFORMATIVE POST

